PLACEMENT GUIDE

Zero to Advanced: How to Find and Get a Job
after BCA, B.Tech or Computer Applications

Skills | Resume | GitHub | LinkedIn | Naukri | Interviews | 90-Day Plan

Contents

#	Section
1	The big picture: what placement really means
2	Job roles you can target
3	Semester-by-semester roadmap
4	Skills: Level 0 to Advanced
5	Data Structures and Algorithms (DSA) plan
6	Projects that get you shortlisted
7	Resume: structure, ATS tips and examples
8	GitHub profile that recruiters trust
9	LinkedIn profile and networking
10	Where to find jobs: Naukri, LinkedIn and more
11	Recruiters: how to find them and message them
12	Selection process: rounds you will face
13	Interview preparation: technical and HR
14	Aptitude, communication and soft skills
15	Fake job alerts and safety
16	Free resources
17	90-day action plan and final checklist

How to use this guide: Read sections 1 to 4 first, then pick the level you are at today (Level 0, 1 or 2) and follow the roadmap. Do not try to do everything at once. Consistency for 90 days beats a single week of panic.

1. The Big Picture

Placement means getting your first job offer, either through your college (**on-campus**) or by applying to companies yourself (**off-campus**). Nobody can guarantee a job, but you can make yourself a strong candidate. Recruiters mainly check five things:

- **Skills:** Can you write code and solve problems?
- **Proof:** Projects, GitHub, certificates, internships.
- **Communication:** Can you explain your work clearly in English or in your working language?
- **Basics:** Aptitude, logical reasoning, CS fundamentals (DBMS, OS, networks, OOP).
- **Attitude:** Willingness to learn, honesty, punctuality.

On-campus vs off-campus

	On-campus	Off-campus
How it works	Companies visit your college or a pool campus	You apply on job portals, referrals, hiring drives
Pros	Easy process, many students hired at once	Unlimited companies, works even if college has no drives
Cons	Limited companies, competitive	You must build the profile and apply consistently
Best strategy	Prepare for both. Do not depend on only one route.	

Golden rule: A good resume gets you the interview. Good fundamentals get you the offer. Start early, apply widely, and keep improving after every rejection.

2. Job Roles You Can Target

Role	What you do	Key skills
Software Developer / Engineer	Build and maintain applications	DSA, one language (Java/Python/C++), OOP, DBMS, Git
Web Developer (Frontend / Backend / Full-stack)	Build websites and web apps	HTML, CSS, JavaScript, React, Node.js or PHP/Laravel, SQL
QA / Software Tester	Test software and find bugs	Testing basics, SQL, Selenium, API testing (Postman)
Data Analyst	Analyse data, build dashboards	Excel, SQL, Python (pandas), Power BI or Tableau, statistics
Technical Support / System Admin	Fix issues, manage systems and servers	Networking, Linux, Windows, troubleshooting, communication
DevOps / Cloud (entry level)	Deploy and monitor applications	Linux, Git, Docker, CI/CD, AWS/Azure basics
Cyber Security (entry level)	Protect systems and find vulnerabilities	Networking, Linux, security basics, tools like Wireshark
Mobile App Developer	Build Android/iOS apps	Kotlin/Java or Flutter/React Native, APIs

Tip: Do not chase every role. Pick **one main track** (for example Web Development) and one backup (for example Testing or Data Analysis). Depth beats breadth.

3. Semester-by-Semester Roadmap

Stage	Focus	Goal by end of stage
Year 1 (Sem 1-2)	C programming, logic building, computer basics, typing and English reading, Excel basics	Comfortable writing small programs; create GitHub account
Year 2 (Sem 3-4)	OOP with Java or C++/Python, DBMS + SQL, HTML/CSS/JS, start DSA	1 or 2 small projects; 100+ easy/medium coding problems
Year 3 / Final year (Sem 5-6)	Advanced DSA, a full project, OS/CN basics, resume, mock interviews, apply	Internship or strong project, polished resume, active applications
BTech extra (Year 4)	System design basics, larger project, competitive coding, internship	Offer or multiple interview calls

If you are already in final year and started late: Skip perfection. Go straight to the 90-day plan in Section 17: fix your resume, learn one language deeply, solve 150 core DSA problems, build one solid project, and apply every day.

4. Skills: Level 0 to Advanced

Level 0: Beginner (starting from zero)

- Learn **one** programming language first: C, Python or Java. Understand variables, loops, functions, arrays, strings.
- Basic computer skills: file management, typing, browser use, Excel/Google Sheets.
- Logic building: solve 30 to 50 pattern and basic problems.
- Create accounts: GitHub, LinkedIn, LeetCode/HackerRank.
- Improve English: read one short article daily, speak your own introduction aloud.

Level 1: Intermediate

- **OOP:** classes, objects, inheritance, polymorphism, encapsulation, abstraction.
- **DBMS + SQL:** joins, group by, subqueries, normalization, keys, transactions.
- **Web basics:** HTML, CSS, JavaScript, plus a backend (Node.js/Express, PHP, Django or Spring Boot).
- **Git and GitHub:** commit, branch, merge, pull request.
- **DSA:** arrays, strings, recursion, sorting, searching, linked list, stack, queue, hashing.
- **CS fundamentals:** OS (process, thread, deadlock, scheduling), Computer Networks (OSI, TCP/IP, HTTP, DNS).

Level 2: Advanced (job-ready)

- **DSA:** trees, graphs, heaps, dynamic programming, greedy, backtracking, two pointers, sliding window.
- **Full project** with authentication, database, REST APIs, deployment.
- **System design basics:** load balancer, caching, database indexing, REST vs GraphQL (mainly for B.Tech / product companies).
- **Tools:** Postman, Docker basics, Linux commands, cloud basics (AWS/Azure free tier).
- **Interview skills:** explain your approach aloud, dry-run code, discuss time and space complexity.
- **Open source or internship** experience to stand out.

5. Data Structures and Algorithms (DSA) Plan

Many companies (especially product companies and mass recruiters' higher-paying roles) test DSA. Learn the pattern, not the answer.

Week	Topics	Practice target
1-2	Time and space complexity, arrays, strings	30 problems
3	Recursion, basic maths, hashing (HashMap/HashSet)	20 problems
4	Sorting, binary search, two pointers, sliding window	25 problems
5	Linked list, stack, queue	25 problems
6	Trees and binary search trees	25 problems
7	Heaps, greedy, backtracking basics	20 problems
8	Graphs (BFS, DFS, shortest path basics)	20 problems
9-10	Dynamic programming basics	20 problems
11-12	Revision + timed mock tests	Solve mixed sets in 45 to 60 minutes

- Solve problems on **LeetCode, GeeksforGeeks, HackerRank, CodeChef** or Codeforces.
- Use a curated list (for example NeetCode 150 or Striver's SDE Sheet) so you do not waste time choosing problems.
- Rule: try for 20 to 30 minutes first, then read the solution, then re-solve it yourself the next day.
- Keep a notebook of patterns and mistakes.

6. Projects That Get You Shortlisted

Projects are your **proof**. Recruiters prefer 2 to 3 good, working projects over 10 copied ones. Every project should have a GitHub repo, a README with screenshots, and (if web) a live link.

Level	Project ideas	What it shows
Beginner	Calculator, To-do app, Quiz app, Portfolio website, Weather app using an API	Basics, UI, API calls
Intermediate	Library or Student management system, Blog with login, E-commerce cart, Expense tracker	Database, CRUD, authentication, roles
Advanced	Full college management system with admin panel, student portal and result module; chat app; job portal; hospital or booking system; AI-powered app using an API	Full-stack skills, database design, real-world thinking, deployment

What makes a project strong

- Solves a real problem (college, shop, clinic, local business).
- Has login and roles (admin / user), a proper database and validation.
- Clean code, folder structure, and comments; no hard-coded passwords in the repo.
- README: problem, features, tech stack, setup steps, screenshots, live demo link.
- Deployed online (free options: Netlify, Vercel, Render, GitHub Pages) when possible.
- You can explain every line and every design decision in an interview.

Interview truth: If you cannot explain your own project deeply (database tables, why you chose the tech, what bug you fixed), it can hurt more than help. Build fewer projects, but know them completely.

7. Resume: Structure, ATS Tips and Examples

Fresher resume = **1 page**, clean, no photo (unless asked), PDF format, file name like *Firstname_Lastname_Resume.pdf*.

Order of sections

- **Header:** Name, phone, professional email, city, LinkedIn URL, GitHub URL, portfolio link.
- **Summary** (2 lines): your target role and top skills.
- **Skills:** Languages, Web, Databases, Tools (only what you can discuss).
- **Projects** (2 to 3): name, tech stack, 2 to 3 impact bullets, GitHub/live link.
- **Internships / Experience** (if any).
- **Education:** degree, college/university, year, CGPA/percentage.
- **Certifications and Achievements:** only relevant ones.

How to write a good bullet

Formula: **Action verb + what you built + technology + result**.

Weak	Strong
Made a college project.	Built a College Management System using PHP and MySQL with role-based login for students and admin, managing profiles, notices and results.
Worked on website.	Developed a responsive student portal (HTML, CSS, JavaScript) with dynamic session and attendance display fetched via SQL joins.
Good at coding.	Solved 200+ DSA problems on LeetCode and GeeksforGeeks; strong in arrays, strings and hashing.

ATS (Applicant Tracking System) tips

- Use simple fonts (Arial, Calibri) and standard headings. Avoid tables inside tables, images and fancy graphics.
- Copy important keywords from the job description (for example Java, SQL, REST API) if you truly know them.
- Never lie about skills. Interviewers ask about everything on your resume.
- Proofread. Spelling mistakes are a fast rejection.
- Keep separate versions: one for Web Developer, one for Software Developer, one for Data roles.

8. GitHub Profile That Recruiters Trust

- Use a professional username (your name), a clear photo and a short bio: *"BCA student | Web developer | PHP, MySQL, JavaScript"*.
- Create a **profile README** (repository with the same name as your username): who you are, skills, top projects, contact.
- **Pin** your 4 to 6 best repositories.
- Every repo needs a README with screenshots and setup steps. Add a licence and .gitignore.

- Commit regularly with meaningful messages ("Add login validation", not "update").
- Never upload passwords, API keys or database credentials. Use environment/config files that are ignored by Git.
- Contribute to open source: start with documentation fixes and "good first issue" labels.
- Green squares are not everything, but steady activity shows discipline.

9. LinkedIn Profile and Networking

Profile checklist

- **Headline:** not just "Student". Example: *"BCA Student | Full-Stack Developer (PHP, MySQL, JS) | Open to Internships and Entry-Level Roles"*.
- Clear, friendly profile photo and a simple banner.
- **About:** 4 to 6 lines on your skills, what you are building and what you are looking for.
- Add Projects, Skills, Education, Certifications; ask seniors or teachers for a recommendation.
- Turn on **Open to Work** (recruiters-only visibility is possible) and set your target roles and locations.
- Add your GitHub and portfolio links in the Featured section.

Networking that works

- Connect with **alumni** from your college who work in IT, HR/recruiters, and professionals in your target role.
- Post or comment about what you are learning or building once or twice a week. It builds visibility.
- Follow company pages you like; many post openings and referral news.
- Ask for **advice** and referrals politely; never demand a job.

10. Where to Find Jobs: Naukri, LinkedIn and More

Platform	Best for	How to use it well
Naukri.com	Wide range of jobs in India, many fresher and IT roles	Complete profile to 100%, upload resume, add key skills, update the profile weekly so it stays fresh in recruiter searches, set job alerts, apply to matching jobs daily
LinkedIn Jobs	IT companies, startups, MNCs; recruiter messages	Use the Easy Apply filter and 'Entry level' filter; set alerts; message the recruiter or hiring manager after applying
Indeed / Foundit / Glassdoor	More listings, company reviews	Use Glassdoor for salary and interview experiences
Internshala	Internships and fresher jobs	Great for first internship; apply with a short, personal cover message
Wellfound (AngelList) / Instahyre / Cutshort	Startups and tech-focused hiring	Create a profile with projects and GitHub; startups value skills over marks
Company career pages	TCS, Infosys, Wipro, Cognizant, Accenture, Capgemini, HCL, Tech Mahindra and others	Most large companies run fresher hiring drives; check the official careers page and register there
College placement cell / alumni	On-campus drives and referrals	Register early, keep documents ready, attend pre-placement talks
GitHub, LeetCode, HackerRank, CodeChef	Proving skills	Recruiters and companies sometimes hire through coding contests and skill tests
Referrals	Highest success rate	Ask alumni or seniors for referrals; a good referral often gets your resume looked at first

Daily job-search routine (30 to 45 minutes)

- Check alerts on Naukri and LinkedIn; apply to 5 to 10 roles that fit at least 60 to 70 percent of the requirements.
- Track every application in a sheet: company, role, date, link, status, follow-up date.
- Send 2 to 3 connection or referral messages.
- Refresh your Naukri profile (edit a small detail) so it stays active.

11. Recruiters: How to Find Them and Message Them

Finding the right people

- On LinkedIn search: *"Recruiter"*, *"Talent Acquisition"* or *"HR"* + company name or + your city.
- Look for people who recently posted "We are hiring" and comment or message them.
- Check the job post itself; many list the recruiter or hiring manager.
- Well-maintained recruiter profiles usually have a clear photo, company name, a headline like "Talent Acquisition at X", regular hiring posts and good connections.

Message template: LinkedIn connection note (short)

Hello [Name], I am a BCA student skilled in PHP, MySQL and JavaScript, and I have built a college management system. I saw your opening for [Role]. I would be grateful if you could review my profile. Thank you!

Message template: email to recruiter

Subject: Application for Junior Web Developer: [Your Name]

Dear [Name],
I am [Your Name], a BCA graduate from [College/University]. I am applying for the [Role] position. I have built projects using PHP, MySQL and JavaScript (GitHub: [link]). My resume is attached. I would welcome a chance to discuss how I can contribute to your team.

Regards,
[Name] | [Phone] | [LinkedIn]

- Keep it short, specific and polite. Attach the PDF resume.
- Follow up once after 5 to 7 days. Do not spam.
- Never pay any recruiter or agency for a job (see Section 15).

12. Selection Process: Rounds You Will Face

Round	What is tested	How to prepare
Resume shortlisting	Skills match, marks cut-off, projects	Tailor the resume; keep it clean and honest
Online test / Aptitude	Quantitative aptitude, logical reasoning, verbal ability, sometimes basic coding or CS MCQs	Practise daily on IndiaBix, PrepInsta or similar; take timed tests
Coding round	1 to 3 problems on arrays, strings, hashing and so on	Solve 150+ DSA problems; practise writing code without hints
Technical interview	Your projects, OOP, DBMS/SQL, OS, CN, coding on paper or screen	Revise fundamentals; know your projects deeply
Managerial / HR round	Communication, attitude, goals, salary, relocation, shifts, bond	Prepare honest, confident answers; research the company
Offer / Documents	Offer letter, background verification	Keep marksheets, ID proof and certificates ready and consistent

13. Interview Preparation: Technical and HR

Technical topics to revise

Area	Commonly asked
Programming / OOP	Four pillars of OOP, abstract class vs interface, overloading vs overriding, constructors, exceptions, collections (for Java), pointers (for C/C++)
DBMS / SQL	Primary vs foreign key, normalization (1NF to BCNF), joins, ACID properties, indexing, GROUP BY / HAVING, write queries live
Operating Systems	Process vs thread, scheduling, deadlock and its conditions, memory management, paging
Computer Networks	OSI vs TCP/IP, HTTP vs HTTPS, DNS, IP address, TCP vs UDP
Web	How a request works, GET vs POST, cookies vs sessions, REST API, SQL injection and XSS basics
Your projects	Why this tech, DB schema, hardest bug, what you would improve, how you handled security (password hashing, validation)

Common HR questions

- Tell me about yourself. (60 to 90 seconds: education, skills, projects, goal.)
- Why do you want to join our company? (Research the company; give a real reason.)
- What are your strengths and weaknesses? (Be honest, show how you are improving.)
- Where do you see yourself in 5 years?
- Are you comfortable with relocation, night shifts or a service agreement?
- Why should we hire you?
- Do you have any questions for us? (Always ask 1 or 2: team, training, growth.)

The STAR method for behavioural questions

Situation, **T**ask, **A**ction, **R**esult. Example: a time you solved a difficult bug or managed a team project deadline.

Mock interviews

- Practise with friends, seniors or in front of a mirror; record yourself.
- Talk aloud while solving coding problems: explain the idea, then code, then test with an example.
- If you do not know an answer, say so honestly and explain how you would find it.

14. Aptitude, Communication and Soft Skills

- **Aptitude:** percentages, profit and loss, time and work, ratios, averages, number series, permutations, puzzles. Practise 20 questions a day.
- **Verbal:** reading comprehension, grammar, sentence correction. Read newspapers or tech blogs daily.
- **Communication:** speak simple, correct English. Prepare a self-introduction and answers to your project questions in English.
- **Email etiquette:** clear subject, greeting, short body, polite closing.
- **Body language:** eye contact, calm voice, sit straight, smile. Dress neatly for interviews (formal or smart casual).

- **Time management:** join calls 10 minutes early and test your internet, camera and microphone for online interviews.

15. Fake Job Alerts and Safety

> **Warning:** Genuine companies **never** ask you to pay money for a job, interview, training kit, laptop, ID card or "security deposit". If someone asks, it is a scam.

- Verify the company: official website, LinkedIn page, Glassdoor reviews, and the recruiter's email domain (for example name@company.com, not a free email address).
- Be careful with offers that come without any interview, or that promise very high salaries for freshers.
- Do not share Aadhaar, PAN or bank details before you have an official offer and joining process.
- Do not click unknown links or install unknown apps for "verification".
- Read the offer letter fully: role, salary structure (CTC vs in-hand), joining date, probation, notice period, service bond.

16. Free Resources

Purpose	Resources
Programming basics	freeCodeCamp, CS50, W3Schools, MDN Web Docs, Programiz; Hindi options include Apna College and CodeWithHarry on YouTube
DSA	NeetCode, Striver's SDE Sheet (take U Forward), GeeksforGeeks, LeetCode, Abdul Bari (algorithms videos)
SQL	SQLBolt, W3Schools SQL, HackerRank SQL, LeetCode Database section
Roadmaps	roadmap.sh for Frontend, Backend, DevOps, Data Analyst and more
CS fundamentals	GeeksforGeeks, Gate Smashers (DBMS, OS, CN), NPTEL courses
Aptitude	IndiaBix, PrepInsta, GeeksforGeeks aptitude section
Mock interviews	Pramp, Interviewing.io, peers and seniors
Government / free skilling	NPTEL, SWAYAM, Skill India Digital; check for free certificates and internships
Portfolio hosting	GitHub Pages, Netlify, Vercel, Render (free tiers)

Tip: Choose one course per topic and finish it. Collecting resources without finishing them is the most common trap.

17. 90-Day Action Plan and Final Checklist

Days	Focus	Deliverable
1-15	Pick your target role. Fix basics in one language. Create resume v1, GitHub and LinkedIn profiles. Start 2 DSA problems a day.	Resume v1, profiles live, 30 problems
16-45	DSA core topics, SQL and CS fundamentals. Build or improve one main project. Start aptitude practice.	Project on GitHub with README, 80 to 100 problems
46-70	Advanced DSA (trees, graphs, DP basics). Deploy project. Update resume v2. Begin applying (5 jobs per day). Message recruiters and alumni.	Live demo link, 30+ applications, referral requests sent
71-90	Mock interviews, aptitude tests, HR preparation. Revise projects and fundamentals. Continue applying and follow up.	5+ mock interviews, interview calls in progress

Final checklist

- [ ] One main programming language I can code in comfortably
- [ ] 150+ DSA problems solved and patterns noted
- [ ] SQL, OOP, OS and CN basics revised
- [ ] 2 to 3 projects with GitHub README and (if possible) live links
- [ ] One-page ATS-friendly resume in PDF
- [ ] GitHub profile README and pinned repos
- [ ] LinkedIn profile complete with headline, About and Featured links
- [ ] Naukri profile 100% complete, alerts on, updated weekly
- [ ] Job tracker sheet started, daily applications habit
- [ ] 5+ mock interviews done, self-introduction ready
- [ ] Documents ready: marksheets, ID proof, passport-size photos

Remember: Rejections are part of the process, not a verdict on you. Review what went wrong, fix one thing, and apply again. Small daily effort adds up: two problems, five applications and one good message every day can change your year. Best of luck!